

Министерство образования и науки Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
“ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО”

Факультет программной инженерии и компьютерной техники (ПИКТ)

Направление подготовки (специальность) – 09.04.04 (Нейротехнологии и программная инженерия)

Информатика

Лабораторная работа № 3

Выполнил студент

Попов Глеб Ильич

Группа № Р3121

Преподаватель: Болдырева Елена Александровна

г. Санкт-Петербург

2024 г.

Оглавление

<i>Задание 1</i>	3
.....	3
<i>Решение 1</i>	4
<i>Задание 2</i>	5
<i>Решение 2</i>	6
<i>Задание 3</i>	7
<i>Решение 3</i>	8
<i>Вывод</i>	12
Список литературы:	12

Задание 1

Задание 1. (20% из 100)

- 1) Реализуйте программный продукт на языке Python, используя регулярные выражения по варианту, представленному в таблице.
- 2) Для своей программы придумайте минимум 5 тестов. Каждый тест является отдельной сущностью, передаваемой регулярному выражению для обработки. Для каждого теста необходимо самостоятельно (без использования регулярных выражений) найти правильный ответ. После чего сравнить ответ, выданный программой, и полученный самостоятельно.
- 3) Программа должна считать количество смайликов определённого вида (вид смайлика описан в таблице вариантов) в предложенном тексте. Все смайлики имеют такую структуру:

[*глаза*][*нос*][*рот*].

Вариантом является различные наборы глаз, носов и ртов.

Номер в ИСУ % 5	Глаза	Номер в ИСУ % 4	Нос	Номер в ИСУ % 7	Рот
0	:	0	-	0	(
1	;	1	<	1	)
2	X	2	-{	2	O
3	8	3	<{	3	
4	=			4	\
				5	/
				6	P

Рис 1. Задание 1

```
import re

5 usages
def smiley_count(text):
    # Используем raw string (r) для экранирования фигурных и круглых скобок
    pattern = r"=-\{ \( " # \{ и \( экранируют фигурные и круглые скобки
    return len(re.findall(pattern, text))

test1 = "=-{( =-{( :( =-{( =-{( :( :( :( :( :("
test2 = " :) :( =-{( :( :( :( =-{( :( :( :( :( :( =-{("
test3 = "=-{( =-{( =-{( =-{("
test4 = ":( :( :( :( :( :("
test5 = "=-{( =-{( :) :( =-{( :( :( :( :( :( :( :("

print(smiley_count(test1))
print(smiley_count(test2))
print(smiley_count(test3))
print(smiley_count(test4))
print(smiley_count(test5))
```

lab3 x

C:\Users\gleby\PycharmProjects\pythonProject31\.venv\Scripts\python.exe C:\Users\g

4
3
4
0
3

Рис 2. Ответ задание 1.

Задание 2. (40% из 100)

- 1) Реализуйте программный продукт на языке Python, используя регулярные выражения по варианту, представленному в таблице.
- 2) Для своей программы придумайте минимум 5 тестов.
- 3) Протестируйте свою программу на этих тестах.

0

Написать регулярное выражение, которое проверяет корректность email и в качестве ответа выдаёт почтовый сервер (почтовый сервер – часть email идущая после «@»). Для простоты будем считать, что почтовый адрес может содержать в себе буквы, цифры, «.» и «_», а почтовый сервер только буквы и «.». При этом почтовый сервер, обязательно должен содержать верхний уровень домена («.ru», «.com», etc.)

Пример:

Ввод	Вывод
students.spam@yandex.ru	yandex.ru
example@example	Fail!
example@example.com	example.com

Рис 3,4. Задание 2

Решение 2

```
import re

5 usages
def email_server(email):
    pattern = r'^[\w.-]+@([a-zA-Z.-]+\.[a-z]{2,})$'
    match = re.match(pattern, email)
    return print(match.group(1)) if match else print("Fail!")

1 usage
def test_email_server():
    print("Test 1: ", end="")
    mailto: email_server("user@example.com")
    print("Test 2: ", end="")
    mailto: email_server("john.doe@sub.domain.org")
    print("Test 3: ", end="")
    email_server("invalid-email@")
    print("Test 4: ", end="")
    email_server("no-at-symbol.com")
    print("Test 5: ", end="")
    email_server("admin@localhost")

test_email_server()
```

Рис 5. Ответ задание 2.

Задание 3. (40% из 100)

1. Определить номер варианта как остаток деления номера в ИСУ на 36. В случае, если в данный день недели нет занятий, то увеличить номер варианта на восемь.
2. Изучить форму Бэкуса-Наура.
3. Изучить особенности протоколов и форматов обмена информацией между системами: JSON, YAML, XML.
4. Понять устройство страницы с расписанием для своей группы: <https://itmo.ru/ru/schedule/0/P31110/schedule.htm>
5. Исходя из структуры расписания конкретного дня, сформировать файл с расписанием в формате, указанном в задании в качестве исходного.
6. **Обязательное задание:** написать программу на языке Python 3.x, которая бы осуществляла парсинг и конвертацию исходного файла в новый.
7. Нельзя использовать готовые библиотеки, в том числе регулярные выражения в Python и библиотеки для загрузки XML-файлов.
8. **Дополнительное задание №1** (позволяет набрать +10 процентов от максимального числа баллов БаРС за данную лабораторную).
 - a) Найти готовые библиотеки, осуществляющие аналогичный парсинг и конвертацию файлов.
 - b) Переписать исходный код, применив найденные библиотеки. Регулярные выражения также нельзя использовать.
 - c) Сравнить полученные результаты и объяснить их сходство/различие.

Решение 3

```
def parseyamlfunction(yaml_data):  
  
    #Самописный YAML-парсер для обработки расписания.  
  
    lines = yaml_data.strip().split("\n") # Разбиваем входные данные по строкам, чтобы обработать каждую по отдельности  
  
    schedule = {"day": None, "classes": []} # Создаем словарь для итогового расписания: день недели и список занятий  
  
    currentclass = None# Переменная для хранения текущего занятия  
  
    # Обработываем каждую строку YAML-файла  
    for line in lines:  
  
        if line.strip().startswith("day:"): # Если строка описывает день недели, извлекаем его  
            schedule["day"] = line.split(":")[1].strip().strip('\"')  
  
        elif line.strip().startswith("- time:"): # Если строка начинается с описания времени, создаем новое занятие  
            if currentclass:  
                schedule["classes"].append(currentclass) # Если предыдущее занятие еще не добавлено, добавляем его  
                currentclass = {"time": line.split(":")[1].strip().strip('\"')}  
            # Если строка относится к текущему занятию, добавляем ключ-значение  
            elif currentclass is not None:  
                key,value = line.split(":", 1) # Разделяем строку по первому двоеточию  
                currentclass[key.strip()] = value.strip().strip('\"')  
  
    # После завершения цикла добавляем последнее занятие, если оно существует  
    if currentclass:  
        schedule["classes"].append(currentclass)  
  
    # Возвращаем итоговый словарь с расписанием  
    return schedule
```

1 usage

1 usage

```
def xmlfunction(schedule):|
```

```
#Генерация XML-кода на основе распарсенных данных.
```

```
# Начинаем формирование XML-структуры
```

```
xml = "<schedule>\n"
```

```
# Добавляем день недели
```

```
xml += f" <day>{schedule['day']}</day>\n"
```

```
# Добавляем секцию занятий
```

```
xml += " <classes>\n"
```

```
for c in schedule["classes"]:
```

```
    # Для каждого занятия открываем тег <class>
```

```
    xml += " <class>\n"
```

```
    for key, value in c.items():
```

```
        # Добавляем ключ и значение в формате XML
```

```
        xml += f" <{key}>{value}</{key}>\n"
```

```
    # Закрываем тег <class>
```

```
    xml += " </class>\n"
```

```
# Закрываем секцию занятий и весь документ
```

```
xml += " </classes>\n"
```

```
xml += "</schedule>"
```

```
return xml
```

1 usage

```
def parseyamlfunctiontoxmlfuncrion(inputyamlfile, outputxmlfile):  
  
    #Чтение данных из YAML-файла, их обработка и запись в XML.  
  
    # Открываем YAML-файл и читаем содержимое  
    with open(inputyamlfile, "r", encoding="utf-8") as file:  
        yaml_data = file.read()  
  
    # Преобразуем YAML-данные в структуру Python  
    schedule = parseyamlfunction(yaml_data)  
  
    # Создаем XML-документ на основе структуры  
    xml_result = xmlfunction(schedule)  
  
    # Открываем файл для записи XML-результата  
    with open(outputxmlfile, "w", encoding="utf-8") as file:  
        file.write(xml_result)  
  
    # Пример вызова функции для конвертации  
    inputyamlpath = "labinf3.yaml" # Замените на реальный путь к вашему YAML-файлу  
    outputxmlpath = "labinf3.xml" # Укажите путь, куда будет сохранен XML-файл  
  
    # Конвертируем файл из YAML в XML  
    parseyamlfunctiontoxmlfuncrion(inputyamlpath, outputxmlpath)  
  
    # Сообщение об успешной конвертации  
    print(f"XML-файл успешно создан: {outputxmlpath}")
```

Доп задание 1

```

1 import yaml
2 import xml.etree.ElementTree as ET
3 from xml.dom.minidom import parseString
4
5 def parse(input_yaml_file):
6     """
7     Парсинг YAML с использованием PyYAML.
8     """
9     with open(input_yaml_file, "r", encoding="utf-8") as file:
10         return yaml.safe_load(file) # Чтение YAML в виде Python-словаря
11
12 def build_xml(schedule):
13     """
14     Генерация XML с использованием ElementTree.
15     """
16     root = ET.Element("schedule") # Корневой элемент
17     day = ET.SubElement(root, "day") # Элемент для дня недели
18     day.text = schedule["schedule"]["day"]
19
20     classes = ET.SubElement(root, "classes") # Элемент для списка занятий
21     for cls in schedule["schedule"]["classes"]:
22         class_element = ET.SubElement(classes, "class")
23         for key, value in cls.items():
24             element = ET.SubElement(class_element, key)
25             element.text = value
26
27     return root
28
29 def xml(element):
30     """
31     Преобразует ElementTree-объект в форматированный XML-строку.
32     """
33     # Преобразуем элемент в строку
34     rough_string = ET.tostring(element, encoding="utf-8")
35     # Форматируем с помощью minidom
36     reparsed = parseString(rough_string)
37     return reparsed.toprettyxml(indent="  ") # Два пробела как отступ

```

```

39 def file_to_xml(input_yaml_file, output_xml_file):
40     """
41     Парсинг YAML и преобразование в XML с использованием библиотек.
42     """
43     # Чтение YAML
44     schedule = parse(input_yaml_file)
45
46     # Генерация XML
47     xml_element = build_xml(schedule)
48     formatted_xml = xml(xml_element) # Форматирование
49
50     # Сохранение XML в файл
51     with open(output_xml_file, "w", encoding="utf-8") as file:
52         file.write(formatted_xml)
53
54     # Пример вызова
55     input_yaml_path = "timetable2.yaml" # Входной YAML
56     output_xml_path = "timetable2.xml" # Выходной XML
57
58     # Конвертация
59     file_to_xml(input_yaml_path, output_xml_path)
60     print(f"XML-файл успешно создан: {output_xml_path}")
61

```

Вывод

В ходе данной практической работы я научился работать с библиотекой `re` на языке программирования Python, а также изучил структуры YAML и XML-файлов, смог самостоятельно сконвертировать файл `yaml` в `xml`

Список литературы:

Балакишин П.В., Соснин В.В., Машина Е.А. Информатика.— СПб: Университет ИТМО, 2020.— 122 с.

1. Орлов С. А., Цилькер Б. Я. Организация ЭВМ и систем: Учебник для вузов. 2-е изд. – СПб.: Питер, 2011. – 688 с.: ил.
2. Алексеев Е.Г., Богатырев С.Д. Информатика. Мультимедийный электронный учебник.